

Frontend Quiz

JCWD-3302

1. What is the main role of HTML in web development ?
 - a. **To define the structure and content of a web page**
 - b. To style the elements of a webpage
 - c. To execute JavaScript code
 - d. To store data in a database
2. Which HTML element is used to create a hyperlink ?
 - a. **<button>**
 - b. **<link>**
 - c. **<a>**
 - d. ****
3. What does the flex-grow property do in CSS Flexbox ?
 - a. **Controls how much an element should grow to fill available space**
 - b. Defines the initial size of an element
 - c. Specifies if an element can shrink
 - d. Determines the main axis of the flex container
4. What is 'CSS Flexbox' and how is it used to layout elements on a web page?
 - a. **CSS Flexbox is a method used for form validation in CSS.**
 - b. **CSS Flexbox is a method for arranging the layout elements in one dimension (row or column) flexibly**
 - c. **CSS Flexbox is a method for arranging background images within HTML elements.**
 - d. **CSS Flexbox is a method for positioning element layout such as fixed, sticky, and absolute.**
5. What is the core concept of React that enables UI development using reusable pieces ?
 - a. **MVC Pattern**
 - b. **Component-Based Architecture**
 - c. **Layout**
 - d. **Object-Oriented Programming**
6. How do you define a functional component in React ?
 - a. **class MyComponent extends React.Component {}**
 - b. **const MyComponent = new React.Component();**
 - c. **function MyComponent() { return <> </> }**
 - d. **React.createComponent('div', 'Hello')**
7. What is meant by 'props drilling' in the context of React ?
 - a. **Props drilling is a way to organize prop data in a structured and efficient manner**
 - b. **Props drilling is a technique or method for passing props in React**
 - c. **Props drilling is the process of passing props data through multiple layers of component, which can make the code inefficient.**
 - d. **Props drilling is a method for passing props from child component to parent component in react, making the code efficient.**
8. What is the primary advantage of the Virtual DOM in React?
 - a. **It reduces direct updates to the real DOM**
 - b. **It increases the application's size**
 - c. **It makes components larger**
 - d. **It adds compatibility with HTML5**
9. How is useMemo used in React to optimize the performance of a component that fetches data from an external API ?
 - a. **useMemo is used to efficiently fetch data from the API every time the component re-renders.**
 - b. **useMemo is used to optimize performance by memoizing expensive operations or calculations that depend on values derived from API data.**
 - c. **useMemo cannot be used to optimize data fetching from an API.**
 - d. **useMemo is used to manage the rendering of React components.**

10. What is the purpose of the `useState` hook in React?
 - a. **To manage routing**
 - b. **To add state to functional components**
 - c. **To handle component lifecycle**
 - d. **To update the DOM manually**
11. Why is `useReducer` often used for state management instead of `useState` in more complex situations?
 - a. **There is no correct answer.**
 - b. **`useReducer` is simpler than `useState` in its implementation**
 - c. **`useReducer` separates state updates from the component, making it easier to manage complex state.**
 - d. **`useState` separates state updates from the component, making it easier to manage complex state.**
12. Which command is typically used to create a new React + TypeScript project using Vite?
 - a. **`npx create-react-app my-app --template typescript`**
 - b. **`npm init react-typescript my-app`**
 - c. **`npx vite-init react-ts my-app`**
 - d. **`npm create vite@latest my-app -- --template react-ts`**
13. Which hook is used to perform side effects in React?
 - a. **`useMemo`**
 - b. **`useEffect`**
 - c. **`useState`**
 - d. **`useRef`**
14. Which import is correct when using `react-router-dom` in vite reactjs project?
 - a. **`import { Switch, Route } from "react-router-dom";`**
 - b. **`import { BrowserRouter, Routes, Route } from "react-router-dom";`**
 - c. **`import { Router, Route } from "react-router";`**
 - d. **`import { Navigation, Page } from "vite-router-dom";`**
15. What is the correct TypeScript type for a component that receives children?
 - a. **`type Props = { children: JSX.Element };`**
 - b. **`type Props = { children?: string };`**
 - c. **`type Props = { children: React.ReactNode };`**
 - d. **`type Props = { children: Element };`**
16. What is the purpose of defining interfaces or types for props in React TypeScript?
 - a. **To enable static type checking and avoid invalid props**
 - b. **To automatically generate documentation**
 - c. **To improve runtime performance**
 - d. **To prevent React from re-rendering unnecessary components**
17. Which of the following is true about how state is updated in Zustand compared to Redux?
 - a. **Redux uses mutation, while Zustand uses immutable updates**
 - b. **Zustand uses reducers like Redux for all updates**
 - c. **Redux requires pure functions (reducers), while Zustand allows direct state modification via `set`**
 - d. **Zustand uses `dispatch` to trigger actions like Redux**
18. How do you access a React Context using `useContext` ?
 - a. **`const {value, setValue} = useContext(MyContext);`**
 - b. **`const [state, dispatch] = useContext();`**
 - c. **`const context = useContextProvider();`**
 - d. **`useContext(MyContext, state => state.value);`**
19. How do you define a Server Component in Next.js App Router ?
 - a. **By using "use client" at the top of the file**
 - b. **By omitting "use client" at the top of the file**
 - c. **By exporting the component as `ServerComponent`**
 - d. **By wrapping the component with `useServer()`**

20. Why should you avoid copying prop values into the component state?
- Because props values cannot be stored by the state in that component**
 - Because it would create two instance with the same state, which could lead to state desynchronization.**
 - Because the mutation process on state does not allow such mutation processes**
 - Because it would lead to duplicate instances with different state, while state synchronization remains intact but it consume a lot of memory.**
21. What is meant by 'Virtual DOM' in the context of react ?
- A javascript library used to facilitate creating virtual animation interactions in react.**
 - A temporary representation of the actual DOM structure used by React to improve rendering performance.**
 - A type of component in react used to replace structure in the form of functional components**
 - A DOM element used to render components in React.**
22. What is Server Side Rendering (SSR) in react, and what are its benefits?
- SSR is a method for managing global state in react.**
 - SSR is a method for rendering views on the server side before sending them to the client, which allows the application to load faster and have better SEO.**
 - SSR is a method for rendering views only on the client side before sending them to the server, which allows the application to load faster and have better SEO**
 - SSR is a method for rendering only animated views in react before sending them to the server, which allow the applications to load faster and have better SEO**
23. Which of the following statements is correct regarding page navigation in a Next JS project?
- Using Link from next/link is very good for SEO performance because page transition links can be detected by crawlers and have SPA behavior.**
 - There is no correct statement.**
 - Using Link from next/link is good for SEO performance because page transition links cannot be detected by crawlers.**
 - router.push is used for page navigation, which is very good for SEO performance because page transition links can be detected by crawlers and have SPA behavior**
24. Which method is used to fetch data on the server in a Server Component ?
- fetch() inside useEffect**
 - getServerSideProps**
 - fetch() directly inside the component**
 - useSWR**
25. Which URL structure is more SEO-friendly ?
- example.com/page?id=123&sort=asc**
 - example.com/?p=12345**
 - example.com/best-laptops-2024**
 - example.com/index.php?page=home**

26. Here is a functional component in react. How to complete this code with the correct way to manage state and display the state value when the button is pressed.

```
import React, { useState } from "react";
function Counter() {

  return (
    <div>
      <p>Count value</p>
      <button>Increment 1</button>{" "}
    </div>
  );
}
export default Counter;
```

- a. Use `useState` to initialize the count state and `setCount`, and use `setCount` to update the state when the button is pressed
 - b. State is not required in functional component
 - c. Use `state` to initialize the count state and `setState`, and use `setState` to update the state when the button is pressed.
 - d. Use `this.state` to initialize the count state and `this.setState` to update the state when the button is pressed
27. You want to access DOM elements in your react component. How would you use the `useRef` hook to achieve this?

```
import React, { useRef } from 'react';

function ElementManipulation() {
  const elementRef = useRef<HTMLDivElement | null>(null);

  const handleClick = () => {
    // complete code
  };

  return (
    <div>
      <button onClick={handleClick}>Ubah Warna</button>
      <div ref={elementRef} style={{ width: '188px', height: '108px', backgroundColor: 'red' }}></div>
    </div>
  );
}

export default ElementManipulation;
```

- a. You can use `elementRef.style.color = 'blue'` in the `handleClick` function to change the background color of the div element.
- b. You can use `elementRef.current.color = 'blue'` in the `handleClick` function to change the background color of the div element.
- c. You can use `elementRef.style.backgroundColor = 'blue'` in the `handleClick` function to change the background color of the div element
- d. You can use `elementRef.current.style.backgroundColor = 'blue'` in the `handleClick` function to change the background color of the div element.

28. You have a functional component in react that has several hooks

```
import React, { useState, useEffect } from 'react';

function DataFetching() {
  const [data, setData] = useState<any[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    // complete code
  }, []);

  return (
    <div>
      <h1>Data yang diambil:</h1> {/* Corrected heading tag */}
      {loading ? (
        <p>Loading...</p>
      ) : (
        <ul>
          {data.map((item) => (
            <li key={item.id}>{item.title}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

export default DataFetching;

Complete the following code:

- You can directly use data and loading to display data in the UI without needing to use useEffect**
- In useEffect, you will send a HTTP request using fetch to fetch data from the server and populate data with the response. When the data is received, you will change the loading value to false**
- You should add and create a new state inside useEffect to manage data and loading because you cannot use useState outside the return. After that, make an HTTP request directly inside useEffect.**
- You should initialize data and loading with fake data from the start, and then update them when the actual data arrives.**

29. You have a functional component in React that has several hooks

```
import React, { useState, useEffect } from 'react';

function DataFetching() {
  const [data, setData] = useState<any[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    // complete code
  }, []);

  return (
    <div>
      <h1>Data yang diambil:</h1> { /* Corrected heading tag */}
      {loading ? (
        <p>Loading...</p>
      ) : (
        <ul>
          {data.map((item) => (
            <li key={item.id}>{item.title}</li>
          ))}
        </ul>
      )}
    </div>
  );
}

export default DataFetching;
```

Complete the following code:

- You can directly use data and loading to display data in the UI without needing to use `useEffect`/
- In `useEffect`, you will send a HTTP request using `fetch` to fetch data from the server and populate data with the response. When the data is received, you will change the loading value to `false`/
- You should add and create a new state inside `useEffect` to manage data and loading because you cannot use `useState` outside the return. After that, make an HTTP request directly inside `useEffect`.
- You should initialize data and loading with fake data from the start, and then update them when the actual data arrives.

30. What happens when this component is rendered?

```
function ExpensiveCalculation ({ num }) {  
  const result = useMemo (() => {  
    console.log('Calculating... ');  
    return num * 2;  
  }, [num]);  
  
  return <div>Result: {result}</div>;  
}
```

- a. Logs 'Calculating...' on every render
- b. Logs nothing
- c. Logs 'Calculating...' only when num changes
- d. Throws an error

31. What will be the output of this component?

```
type Props = {  
  count?: number;  
};  
  
const Counter = ({ count = 5 }: Props) => {  
  return <p>{count + 1}</p>;  
};
```

- a. It will display 5
- b. It throws a TypeScript error because count is optional
- c. It will display 6
- d. It displays undefined1

32. How do you properly define global state and actions using Zustand?

- a. By using `createSlice` to organize reducers and actions.
- b. By using the `create` function from Zustand to define a store that returns an object with state and actions.
- c. By defining a root reducer and using `combineReducers` to manage state structure.
- d. By using `useReducer` inside React Context to manage state globally.

33. Take a look at the following code! The snippet below is from a Next.js version 14 project that uses the app router.

```
import { useState } from "react";

export default function Home() {
  const [data, setData] = useState<number>(0);

  return (
    <div>
      <button onClick={() => setData(data + 1)}></button>
      <div>{data}</div>
    </div>
  );
}
```

What is wrong with the code ?

- a. You need to define the page using class component .
- b. You need to define the page as client side using "use client" .
- c. The use of useState must be accompanied by useEffect in that component.
- d. The Function in the onClick must be wrapped in seperate function and initialized above before calling it.